AN UNOFFICIAL GUIDE

Inside Skills

Agent Skills and the skills CLI

Covers skills 1.7.4 (tag `v1.7.4`), commit `a80e4c9`.

Unofficial and independently written. Not affiliated with, endorsed by, or reviewed by Vercel or the maintainers of skills.

Contents

	Preface	**5**
	Who this book is for	5
	What this book covers	5
	Versions and environment	5
	How to read this book	6
	Conventions	6
Chapter 1	**Introduction to Agent Skills**	**8**
	Read this chapter if	8
	In this chapter you will	8
	The problem skills solve	8
	What the skills CLI does	9
	The parts of the system	9
	A first skill, end to end	10
Chapter 2	**About Skills and Sources**	**13**
	Read this chapter if	13
	You can skip	13
	What a skill is	13
	The SKILL.md contract	13
	Skill names and install directories	14
	Source formats	15
	Pinning a ref and naming a skill	16
	How the CLI discovers skills	17
	Selecting skills to install	18
Chapter 3	**About Scopes, Agents, and Lock Files**	**19**
	Read this chapter if	19
	In this chapter you will	19
	Project scope and global scope	19
	The canonical skills directory	20
	Universal and agent-specific agents	20
	Symlink mode and copy mode	20
	How the CLI chooses target agents	22
	Lock files	23
	How updates are detected and applied	23

Chapter 4 **Installing and Managing Skills** 25
Before you begin 25
In this chapter you will 25
The sample setup 25
Install a skill into a project 27
List installed skills 28
Install a skill for all your projects 28
Add an agent to an installed skill 28
Check for and apply updates 29
Remove skills 30
Try a skill without installing it 31
Chapter 5 **Sharing Skills Through a Project** 32
Before you begin 32
In this chapter you will 32
Decide what to commit 32
Restore skills from the lock file 33
Sync skills shipped in npm packages 34
Declare skills in package.json 36
Chapter 6 **Authoring and Publishing Skills** 38
Before you begin 38
In this chapter you will 38
Create a skill 38
Write the description and instructions 39
Test a skill before publishing 39
Lay out a repository of skills 40
Publish from a Git repository 40
Publish from a website 40
Ship skills in an npm package 41
Hide work-in-progress skills 42
Chapter 7 **Command Reference** 43
Conventions for every command 43
`skills add` 45
`skills use` 46
`skills list` 46
`skills remove` 47
`skills update` and `skills check` 48
`skills find` 49

`skills init` 49
`skills experimental_install` 49
`skills experimental_sync` 50

Appendix A **Files and Environment Variables** **51**
File locations 51
Project lock file fields 51
Global lock file 52
Environment variables 52

Appendix B **Errors and Recovery** **54**
Errors when adding skills 54
Errors when updating or restoring 55
Unexpected placement and removal results 55

Appendix C **Glossary** **57**

Preface

Who this book is for

This book is for developers who install skills into projects or home directories for one or more coding agents. It also serves those who share skills with a team through a repository, automate installs in scripts or CI, or write skills and publish them for others.

The book assumes you are comfortable with a shell, Git, npm and `npx`, JSON, and YAML frontmatter. You need no prior knowledge of agent skills.

The book states where the CLI writes files. How a particular agent loads, triggers, or runs a skill is defined by that agent, and its own documentation describes it.

What this book covers

The book covers the skill format as the CLI reads it, how a source string is interpreted and searched for skills, and the installation model: scopes, the shared skills directory, agent directories, symlink and copy modes, and lock files. It documents every public command, the workflows for sharing skills through a project, and the ways to publish skills from a Git repository, a website, or an npm package.

It leaves out advice on writing effective instructions, the skills.sh website, and the CLI's internal implementation.

Two commands, `experimental_install` and `experimental_sync`, are experimental. Their names, options, and behavior can change in a later release.

Versions and environment

The book describes skills 1.7.4 (tag `v1.7.4`, commit `a80e4c9`). `npx skills` usually resolves to the newest release, so run `npx skills@1.7.4` when you need output that matches the book.

The CLI requires Node.js 22.20.0 or later. Any source that the CLI clones with Git, including a `file://` URL, also requires Git on your `PATH`.

macOS and Linux follow the same code paths. Where Windows behaves differently, for example by creating directory junctions instead of symbolic links, the text says so and marks the claim **Unverified**, because no example was run on Windows.

Every listing was captured once, from skills 1.7.4 on macOS. Each run used a fresh temporary directory as `HOME`, local fixture skills as sources, and `DISABLE_TELEMETRY=1`, so no run read or changed a real configuration or sent telemetry. No coding agent was detected during capture. Run from inside an agent, some commands select targets differently, as Non-interactive mode and agent detection describes.

How to read this book

If you want to…	Read…	You can skip…
Install skills for yourself now	Chapter 1, then Chapter 4	Chapters 5 and 6
Understand where files go and why	Chapters 2 and 3	Chapter 6
Set up skills for a team repository	Chapters 3, 4, and 5	Chapter 6
Write and publish skills	Chapters 2 and 6	Chapter 5
Look up a flag, file, or error	Chapter 7 and the appendixes	The narrative chapters

Chapter 3, About Scopes, Agents, and Lock Files, is foundational. Read it first if you use more than one coding agent on a machine or in a repository. Its model explains why an install can reach agents you did not name and why removing a skill from one agent can affect another.

Conventions

Commands appear as `npx skills <command>`. Angle brackets mark a placeholder that you replace, brackets included:

```
npx skills add <source> --skill <name> -a <agent> -y
```

Examples run in the sandbox that The sample setup creates. Its root appears as `/tmp/demo`, the project as `/tmp/demo/project`, and `~` means the sandbox home, `/tmp/demo/home`.

Listings omit the banner logo, color codes, spinner frames, and empty `│` gutter lines. They also omit the `Tip:` line that `add` prints when its input is not a terminal. All other output appears as the CLI printed it.

Callouts carry fixed meanings. A NOTE adds context, an IMPORTANT callout marks a condition the procedure depends on, and a WARNING marks an action that can lose files or expose data. A **Current state (skills 1.7.4)** note marks behavior specific to that release, and an **Unverified:** label marks a claim that no run confirmed.

CHAPTER 1

Introduction to Agent Skills

This chapter explains what an agent skill is, what the `skills` CLI does with one, and where the CLI's responsibility ends, then installs one skill into a sample project. If you already know the model and want only the commands, go to Installing and Managing Skills.

Read this chapter if

- You have not used agent skills before, or you have only copied them into place by hand.
- You need to explain to a team what the CLI changes on disk and what it leaves to each agent.

In this chapter you will

- Learn what a skill is and why several agents complicate sharing one.
- Learn what the CLI does and where its responsibility ends.
- Install a skill into a project and list it.

The problem skills solve

A coding agent works from general knowledge plus the instructions it finds in a session. Many tasks need narrower guidance, such as how your team writes release notes or which checks run before a release is tagged. Pasting that guidance into each conversation is unreliable, and copying it into each repository produces versions that drift apart.

A skill packages that guidance as a directory. The directory holds a `SKILL.md` file, which gives the skill a name, a short description, and Markdown instructions, along with any supporting files the instructions refer to. An agent that supports skills reads the descriptions and loads a

skill's instructions when a task calls for them.

Because a skill is only files, it can travel between projects and between agents. The obstacle is location, because agents look for skills in different directories. In a project, Codex and Cursor read `.agents/skills`, while Claude Code reads `.claude/skills`. Without a tool, a team that uses two of these agents keeps two copies of every skill and updates both by hand.

What the skills CLI does

The `skills` CLI, run as `npx skills`, is a package manager for skills. An install does three things:

1. It fetches skills from a source, such as a local folder, a Git repository, or a website.
2. It places each selected skill where the agents you chose will look for it, in the current project or in your home directory.
3. It records where each skill came from in a lock file, so that later commands can update, restore, and remove the skill.

Once the files are in place, the CLI's part is over. It does not run a skill, and it does not decide when an agent uses one; the agent makes that choice. The CLI does not sandbox a skill either. An installed skill acts with the agent's permissions, so if the agent can run shell commands, edit files, or reach the network, a skill's instructions can direct it to do those things.

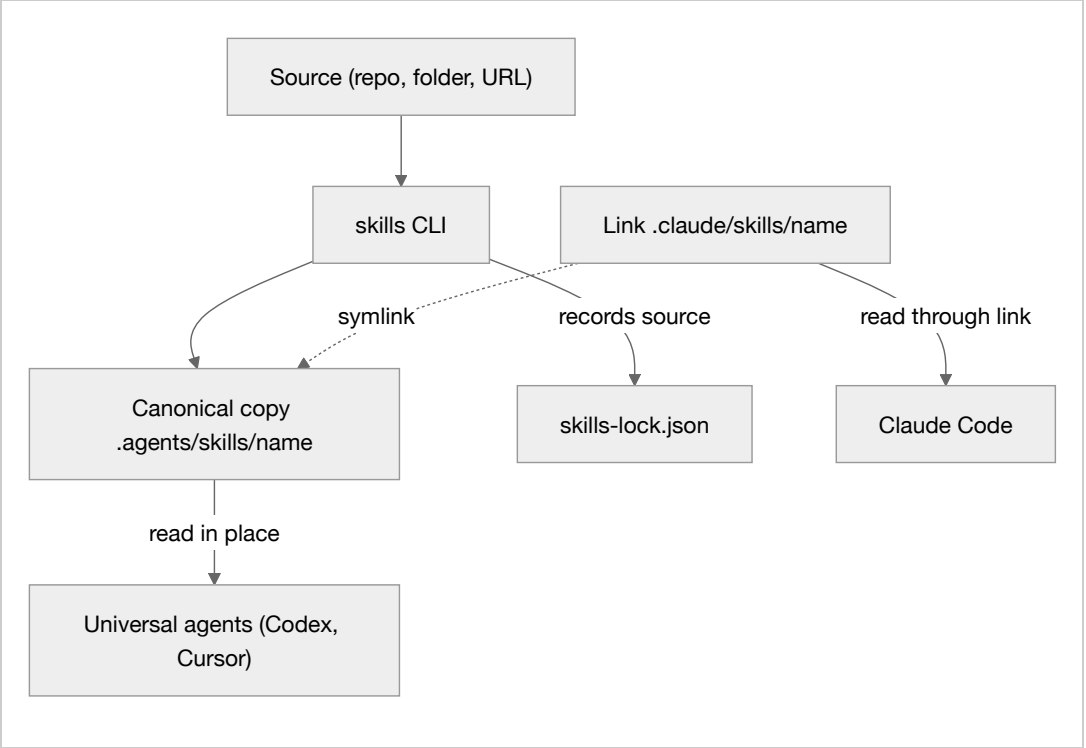
The CLI does not inspect or approve a skill's content. For a public GitHub repository, it can display risk assessments from the skills.sh audit service, but they are advisory, and when the service fails or has no data the install proceeds without them (what the CLI sends).

Responsibility for a skill's content stays with you, which is why a successful install ends with a reminder to review skills before use. Read a skill's `SKILL.md` and supporting files before you install it from a source you do not control.

The parts of the system

An install connects six parts: a source, the CLI, the canonical directory that holds a skill's real files, the agent directories that link to it, the lock file, and the agents that read the result.

Figure 1-1: In symlink mode, the CLI keeps one copy in .agents/skills, and every chosen agent reads it from there.



In symlink mode, the result is one real copy with many readers. When a project installs `release-notes` for Claude Code and Codex, the files exist once, in `.agents/skills/release-notes`. Codex, a universal agent, reads them there, and `.claude/skills/release-notes` is a symbolic link to the same directory. Replacing that one copy changes what every linked agent reads.

A first skill, end to end

This example uses the sandbox from The sample setup: a project at `/tmp/demo/project` and, beside it, a local source at `/tmp/demo/acme-skills` that contains two skills.

From the project directory, install the `release-notes` skill for Claude Code. The `--skill` option selects one of the two skills, `-a claude-code` names the target agent, and `-y` accepts defaults instead of prompting. Name the agent explicitly, because without `-a` the CLI chooses

targets by detecting agents and can install for many more than you intend (How the CLI chooses target agents).

```
$ npx skills add ../acme-skills --skill release-notes -a claude-code -y
┌   skills
◇  Source: /tmp/demo/acme-skills
◇  Local path validated
◇  Found 2 skills
●  Selected 1 skill: release-notes
◇  Installation Summary ──────────────┐
│                                     │
│  ./.agents/skills/release-notes     │
│    copy → Claude Code               │
│                                     │
├─────────────────────────────────────┘
◇  Installation complete
◇  Installed 1 skill ─────────────────────┐
│                                         │
│  ✓ release-notes (copied)               │
│    → ./.claude/skills/release-notes     │
│                                         │
├─────────────────────────────────────────┘
│
└  Done!  Review skills before use; they run with full agent permissions.
```

The project now contains three files:

```
$ find . -type f | sort
./.claude/skills/release-notes/SKILL.md
./.claude/skills/release-notes/template.md
./skills-lock.json
```

Claude Code is the only target, so there is nothing to share and no link to create. The CLI uses copy mode and writes the whole skill directory, supporting file included, straight into `.claude/skills`, although the summary names the canonical path. No `.agents/skills` directory exists.

List what the project has installed:

```
$ npx skills list
Project Skills

release-notes ./.claude/skills/release-notes
  Agents: Claude Code  Source: /tmp/demo/acme-skills
```

Installing and Managing Skills continues from this project with the complete workflow.

CHAPTER 2

About Skills and Sources

This chapter explains what the CLI accepts as a skill, how it reads a source string, and how it chooses skills inside a source. About Scopes, Agents, and Lock Files explains where the files go.

Read this chapter if

- an install skipped a skill, chose the wrong one, or used an unexpected directory name;
- you write skills and need to know what the CLI requires of them.

You can skip

- the rows of the source table for hosts you do not use.

What a skill is

A skill is a directory that contains a file named `SKILL.md`. The file starts with YAML frontmatter that names and describes the skill, followed by Markdown instructions for the agent. The directory can also hold supporting files, which the instructions refer to by relative path.

The CLI installs the whole directory, so a relative reference that works in the source also works after installation.

The SKILL.md contract

The frontmatter must set `name` and `description`, and both must be strings. It must be YAML between two `---` lines at the top of the file. JavaScript frontmatter (`---js`) is not evaluated, so such a file has no fields.

A `SKILL.md` that lacks a required field, sets either field to a non-string value, or contains invalid YAML is skipped with a warning, and discovery continues. The examples in this chapter add two folders to the source from The sample setup: `skills/draft-checklist`, whose frontmatter has only a `name`, and `skills/notes-v2`, whose `name` is `Release Notes v2`.

```
$ npx skills add ../acme-skills --list
…
⚠ Skipped /tmp/demo/acme-skills/skills/draft-checklist/SKILL.md — missing required
frontmatter field(s): description
◇  Found 3 skills
```

The command lists the other three skills and exits with status 0. If no valid skill remains, `add` fails (Errors when adding skills).

Setting `metadata.internal: true` makes a skill internal and hides it from discovery (Selecting skills to install). Hide work-in-progress skills shows the task.

When the CLI copies a skill, it leaves out every `.git`, `__pycache__`, and `__pypackages__` directory and every `metadata.json` file, at any depth.

Skill names and install directories

The CLI names the installed directory after the frontmatter `name`, not after the source folder. It lowercases the name, replaces each run of characters other than `a-z`, `0-9`, `.`, and `_` with a hyphen, trims leading and trailing dots and hyphens, and truncates the result to 255 characters. A name that sanitizes to nothing becomes `unnamed-skill`. The skill in the `notes-v2` folder therefore installs as `release-notes-v2`:

```
$ npx skills add ../acme-skills --skill "Release Notes v2" -a claude-code -y
…
$ ls .claude/skills
release-notes-v2
```

Because the directory comes from the name, two skills with the same name share one install directory, even when their sources differ. Installing the second replaces the first one's files and its lock entry.

Source formats

The CLI tests the source argument of `add` and `use` against these forms from top to bottom and uses the first match.

Form	Example	Recognized by	Source type
Notion	`notion`	`add` only. **Unverified:** also a Notion page URL; fetches skills from Notion	none
Local path	`../acme-skills`	Absolute; starts with `./` or `../`; `.` or `..`; a Windows drive path	`local`
Hosted artifact URL	`https://raw.githubusercontent.com/…/SKILL.md`	Raw, archive, and release-download URLs on GitHub and GitLab	`download`
GitHub URL	`https://github.com/acme/acme-skills/tree/main/skills`	`github.com` repository or tree URL	`github`
GitLab	`gitlab:acme/acme-skills`	`gitlab:` prefix, `gitlab.com` repository URL, or `/-/tree/` URL on any host	`gitlab`
Azure Repos URL	`https://dev.azure.com/acme/app/_git/skills`	A `/_git/<repo>` path on any host	`git`
GitHub shorthand	`acme/acme-skills/skills/release-notes`	`owner/repo`, optional subpath, no `:`; also `github:owner/repo`	`github`
Other HTTP(S) URL	`https://acme.dev/skills`	Any other HTTP(S) URL not ending in `.git`; tried as a well-known index, then downloaded as a `SKILL.md` or archive	`well-known`
Git URL	`git@github.com:acme/acme-skills.git`, `file:///tmp/demo/acme-skills.git`	Anything no row above matches, such as SSH or an HTTP(S) URL ending in `.git`	`git`

> **WARNING**
> Start a relative path with `./` or `../`. Otherwise the CLI reads `my-skills/release-notes` as a GitHub repository and clones it, which can install skills that someone else wrote.

When `GH_HOST` names a GitHub Enterprise host, shorthand sources clone from that host as `git` sources (Environment variables).

Pinning a ref and naming a skill

A source can carry a Git ref (a branch or tag) and a skill name:

- A `#ref` fragment pins the ref on shorthand and on Git, GitHub, GitLab, and Azure Repos sources, as in `acme/acme-skills#v1`. A `file://` URL does not accept one.
- A tree URL (`…/tree/v1/skills`) carries its own ref and subpath.
- `owner/repo@skill` selects one skill by name, and `owner/repo#v1@skill` also pins the ref. The `@skill` part works only on shorthand and Azure Repos sources; other URLs drop it and offer every skill, so use `--skill` with them.

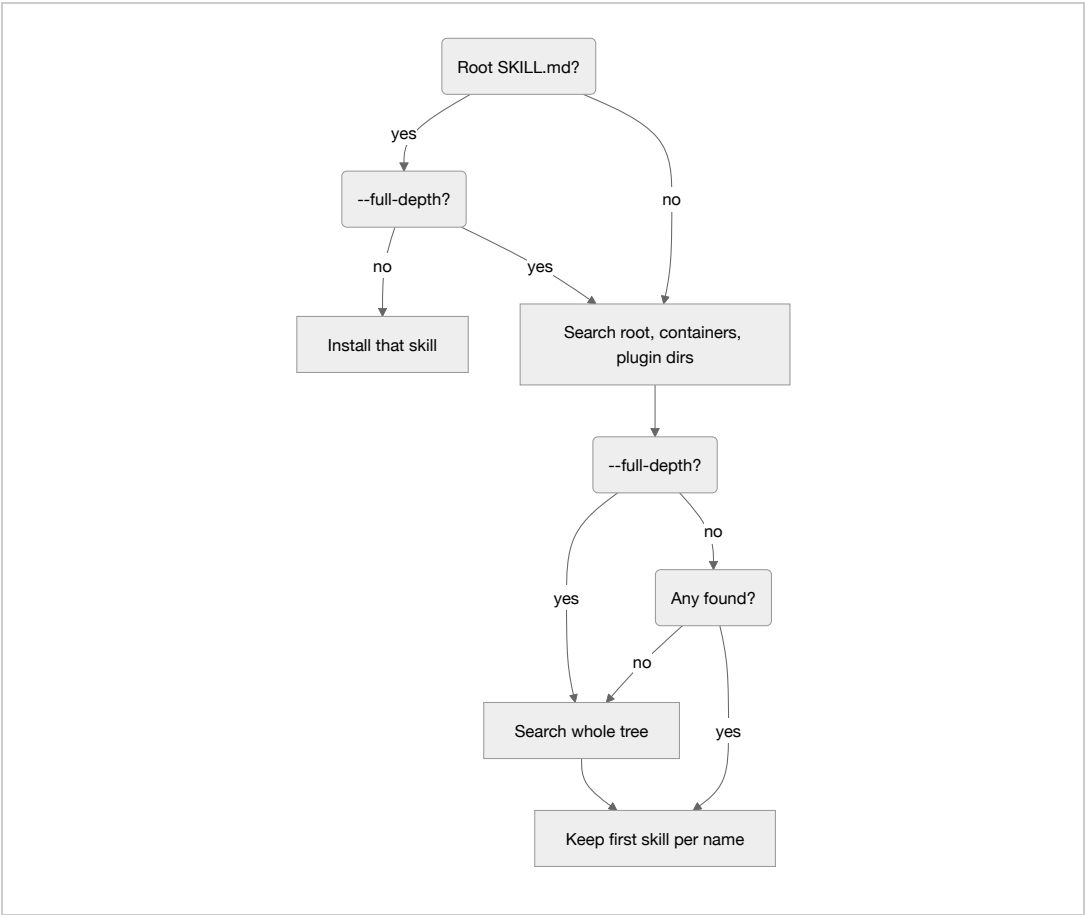
A local path accepts neither form, because the CLI treats `@release-notes` as part of the path:

```
$ npx skills add ../acme-skills@release-notes -a claude-code -y
…
└  Local path does not exist: /tmp/demo/acme-skills@release-notes
```

The command exits with status 1; pass `--skill release-notes` instead.

How the CLI discovers skills

Figure 2-1: A SKILL.md at the top of a source hides everything below it unless you pass --full-depth.



Discovery follows six rules:

1. A subpath makes that directory the search root. A subpath containing `..` is rejected, so the search cannot leave the source.
2. If the search root holds a valid `SKILL.md`, that one skill is the result, unless you pass `--full-depth` (layout advice).

3. Otherwise the CLI checks the immediate subdirectories of the search root, then walks container directories such as `skills/` and `.claude/skills` up to three levels deep (the full list). A directory that holds a skill ends the walk on its branch, so the walk misses skills nested below it.
4. Skill directories declared in `.claude-plugin/marketplace.json` or `.claude-plugin/plugin.json` are searched too.
5. If nothing is found, or you passed `--full-depth`, the CLI searches every directory up to five levels below the search root, skipping `node_modules`, `.git`, `dist`, `build`, and `__pycache__`.
6. When two skills share a name, the first one found wins, and the CLI drops the other without a warning.

Selecting skills to install

Without `--skill` or `@skill`, a source with one skill installs it directly, `--yes` installs every discovered skill, and otherwise the CLI prompts (Non-interactive mode and agent detection).

A `--skill` value selects the skill whose frontmatter `name` matches it exactly, ignoring case. Partial names and sanitized directory names do not match, so `--skill release-notes-v2` finds nothing here. If none of the requested names matches, `add` exits with status 1 and lists the names of the skills it found. If at least one matches, the CLI ignores unmatched names without a warning and exits with status 0. With `--json`, each unmatched name appears as a `skipped` entry and the command exits with status 1.

`--skill '*'` selects every discovered skill. Naming an internal skill makes it visible and installs it, but `'*'` and `--yes` leave internal skills out. Setting `INSTALL_INTERNAL_SKILLS=1` includes them in every discovery.

CHAPTER 3

About Scopes, Agents, and Lock Files

Every install makes the same decisions. The CLI picks a base directory (the scope), picks the agents that receive each skill, and gives each agent either a link to a shared copy or a copy of its own. It then records the install in a lock file, which later drives updates and restores. This chapter explains those decisions so that you can predict which files a command creates.

Read this chapter if

- you share a project or a machine with more than one coding agent;
- you run the CLI from scripts, from CI, or inside an agent session.

In this chapter you will

- tell project scope from global scope and find the canonical copy;
- learn which agents read the canonical directory and which need links;
- follow the order in which the CLI picks target agents;
- see what each lock file records and how `update` uses it.

Project scope and global scope

Project scope is the default, and it writes under the current working directory. The CLI does not search upward for a project root such as a Git repository or an existing `skills-lock.json`. If you run `npx skills add` in `packages/web/`, the skill and a new `skills-lock.json` land in `packages/web/`, even when the repository root already has a lock file. Change to the directory that should own the skills before you run the command.

Global scope (`-g`) writes under your home directory. An interactive run without `-g` or `--yes` asks for a scope when at least one target agent supports global skills. With `--yes` and no `-g`, the scope is the project.

The canonical skills directory

The canonical skills directory is `.agents/skills/` in a project, or `~/.agents/skills/` in global scope. In symlink mode it holds the one real copy of each skill, in a subdirectory named after the skill, such as `.agents/skills/release-notes/`. Skill names and install directories explains how the CLI derives that name.

Universal and agent-specific agents

A universal agent is one whose project skills directory is `.agents/skills`, the canonical directory itself. Codex, Cursor, Gemini CLI, and GitHub Copilot are universal. A universal agent reads the canonical copy directly, so the CLI creates nothing extra for it. An agent-specific agent reads its own directory, such as `.claude/skills` for Claude Code or `.windsurf/skills` for Windsurf, and receives a link to the canonical copy or a copy of its own.

In global scope, the CLI places a universal agent's skills in `~/.agents/skills` and writes nothing into the agent's own home directory. A global install for Cursor creates only `~/.agents/skills/release-notes`, even when `~/.cursor` exists. `CLAUDE_CONFIG_DIR`, `CODEX_HOME`, and similar variables move an agent's home directory (Environment variables).

Eve (`agent/skills`) and PromptScript are project-only. A global install that includes either one reports a per-agent failure, such as `PromptScript does not support global skill installation`, installs for the other agents, and exits with status 0.

The supported agents table, generated from `src/agents.ts`, lists every agent with its `--agent` name and paths.

Symlink mode and copy mode

In symlink mode, the default, the CLI copies the skill once into the canonical directory and links each agent-specific directory to that copy. Links are relative, such as `.claude/skills/release-notes -> ../../.agents/skills/release-notes`, so they still res-

olve after the project is cloned to another path. In copy mode (`--copy`), the CLI writes an independent copy into each target's directory and creates no links.

Figure 3-1: Symlink mode keeps one real copy; copy mode gives each agent its own.



When every target shares one directory, the CLI uses copy mode without `--copy`, because no link is needed. An install for Claude Code alone, or for universal agents alone, therefore reports `(copied)`. If a link cannot be created, the CLI copies the files into that agent's directory and warns `Symlinks failed for:`.

An edit to the canonical copy reaches every linked agent at once. Copies drift apart when you edit one of them. A reinstall deletes and rewrites each directory it targets, which discards local edits.

How the CLI chooses target agents

Without `-a`, the CLI chooses the agents itself. It checks the conditions in the following table in order, and the first one that applies decides the targets.

Condition, checked in order	With –y	Without –y
–a names agents	Those agents	Those agents
Running inside an agent that the CLI maps to a target	That agent + universal	Same (–y is implied)
Eve detected in the project	Eve only	Asks about Eve, then prompts for agents if you decline
One agent detected on the machine	That agent + universal	Same
Several agents detected	Detected + universal	Prompt
No agent detected	Every agent	Prompt

When you pass `-a`, the CLI installs for exactly those agents and adds no universal agents; `-a '*'` selects every agent. Otherwise it checks for an agent session through environment variables (Non-interactive mode and agent detection). If the session maps to none of its agents, such as `AI_AGENT=pi`, `-y` is still implied, so selection continues down the table and, in a clean home, reaches every agent. Next, it looks for Eve: an `agent/` directory in the project plus an `eve` dependency in `package.json`. Failing that, machine detection looks for agent home and config directories such as `~/.claude`.

In symlink mode and project scope, a detected agent-specific agent gets a link only when its top-level directory, such as `.windsurf/`, already exists in the project. Otherwise the result reports `skipped: Windsurf (project directory not found)`, and Windsurf sees nothing until you create the directory and reinstall. The CLI creates the directory anyway for agents you named with `-a` or chose at the prompt, for Claude Code, the one agent the CLI always provisions, and for Eve, which always receives a real copy in `agent/skills`. With `--copy`, the CLI writes a copy into every target's directory and creates directories such as `.windsurf/` that did not exist.

WARNING
Pass `-a` in every script and CI job. Without it, the targets depend on the machine and the environment. On a clean runner, `--yes` installs for every agent and creates `.claude/` and `agent/` in the project, and `--yes --copy` creates more than 50 agent directories there. Inside an agent session, the CLI targets that agent and the universal agents without prompting.

Lock files

The project lock, `skills-lock.json` in the project directory, records project installs so that teammates can restore them and `update` can check them. Commit it. It has `version` 1 and no timestamps, and its entries are sorted by skill name, so branches that add different skills usually merge cleanly. Each entry records the source, the source type, the ref and skill path when they apply, and `computedHash`, a SHA-256 hash of the skill folder's contents. A local source is stored as a path relative to the project, such as `../acme-skills`.

The global lock records `-g` installs, in `~/.agents/.skill-lock.json`, or in `$XDG_STATE_HOME/skills/.skill-lock.json` when that variable is set. It has `version` 3, install and update timestamps, and a `skillFolderHash`, which is the GitHub tree SHA for GitHub sources. The CLI discards a global lock older than version 3, so its skills become updatable only after you reinstall them with `add -g`.

Neither lock records which agents received a skill or which mode was used. Skills installed by direct download from a URL are not in the project lock. Project lock file fields and Global lock file describe the fields.

How updates are detected and applied

`npx skills update` compares lock entries with their sources (`skills update` and `skills check` gives the scope rules). In project scope, it clones each recorded Git source, hashes each skill folder, and compares the result with `computedHash`. In global scope, it compares GitHub tree SHAs for GitHub sources, cloning when the API is unavailable. Local-path and npm-package entries are never updated in place.

For each changed skill, the CLI runs `add <source> --skill <name> -y` in a child process, adding `-g` for global entries. The child chooses agents by the selection rules as they apply at update time, so placement can change. With no agents detected, a skill copied with `-a`

`claude-code` comes back as a link to a new copy in `.agents/skills`, and `agent/skills` appears. Rerun `add` with your `-a` list to restore a placement. A skill that disappeared from its source is reported as deleted upstream.

CHAPTER 4

Installing and Managing Skills

This chapter's sections share one sandbox and build on each other, so run them in order the first time. Read Chapter 3 first if you share a machine or repository with other agents, because its model explains where each command writes and why a removal can reach agents you did not name.

Before you begin

- Node.js 22.20.0 or later.
- Git on your `PATH`.
- The sandbox, created in The sample setup.
- A terminal that is not running inside a coding agent, because inside one the CLI can choose other targets (Non-interactive mode and agent detection).
- Recommended: About Scopes, Agents, and Lock Files.

In this chapter you will

- install, list, update, and remove skills in a project and in your home directory;
- give an installed skill to another agent;
- try a skill without installing it.

The sample setup

The book's examples run in a sandbox at `/tmp/demo`. Its `home/` directory replaces your home directory for the session, so global installs and Git settings stay out of your real one. Open a new shell, delete any `/tmp/demo` left from an earlier run, and run:

```
mkdir -p /tmp/demo/home /tmp/demo/project
export HOME=/tmp/demo/home DISABLE_TELEMETRY=1
git config --global user.name "Acme Docs"
git config --global user.email docs@acme.example
cd /tmp/demo
mkdir -p acme-skills/skills/release-notes acme-skills/skills/changelog-lint
cat > acme-skills/skills/release-notes/SKILL.md <<'EOF'
---
name: release-notes
description: Draft release notes from merged pull requests. Use when preparing a
release or a changelog entry.
---

# Release notes

1. List the pull requests merged since the last release tag.
2. Group them under the headings in template.md.
3. Write one plain sentence per change, naming the user-visible effect.
EOF
printf '## Added\n## Changed\n## Fixed\n' > acme-skills/skills/release-
notes/template.md
cat > acme-skills/skills/changelog-lint/SKILL.md <<'EOF'
---
name: changelog-lint
description: Check CHANGELOG.md entries for format problems. Use before tagging a
release.
---

# Changelog lint

Report entries that lack a version heading, a date, or a category.
EOF
git -C acme-skills init -q
git -C acme-skills add .
git -C acme-skills commit -q -m "Add release-notes and changelog-lint"
git clone -q --bare acme-skills acme-skills.git
cd project
```

The listing creates `acme-skills`, a local source with two skills, and `acme-skills.git`, a bare Git copy for the update examples, and leaves the shell in `project/`. Close the shell when you finish, so that later commands use your real home directory.

Install a skill into a project

The Acme team uses Claude Code and Codex. Create both home directories so that the CLI detects both agents; later sections use Codex detection when listing and removing. Install for Claude Code first with `npx skills add`, the command that A first skill, end to end runs with its full output:

```
mkdir -p ~/.claude ~/.codex
npx skills add ../acme-skills --skill release-notes -a claude-code -y
```

The result reports `release-notes (copied)`, because one target directory means copy mode. Verify what the command wrote:

```
ls .claude/skills/release-notes
cat skills-lock.json
```

```
SKILL.md
template.md
{
  "version": 1,
  "skills": {
    "release-notes": {
      "source": "../acme-skills",
      "sourceType": "local",
      "computedHash":
"807f3bbaa62d61c70f82fda6205bb7f4cba55591e7748a8f2137377e2a6b504d"
    }
  }
}
```

The project lock file records the source relative to the project and a hash of the skill's files (Lock files).

> **WARNING**
> Read a skill's `SKILL.md` and supporting files before you install it. The CLI does not check what they say, and an agent that loads the skill follows its instructions with the agent's own permissions, which can include running commands and editing files (What the skills CLI does).

Install failures are listed in Errors when adding skills.

List installed skills

`npx skills list` shows the project's skills:

```
Project Skills
release-notes ./.claude/skills/release-notes
  Agents: Claude Code  Source: /tmp/demo/acme-skills
```

The CLI builds the `Agents:` line from the files on disk, because the lock file does not record agents. It checks the skills directory of each agent detected on this machine, and of each agent whose directory exists in the project, and lists an agent only when the skill is there. Codex is detected but not listed, because the skill is not yet in `.agents/skills`, where Codex reads. `-g` lists global skills instead; `skills list` describes `-a` and `--json`.

Install a skill for all your projects

Add `-g` to install under your home directory, where every project on the machine sees the skill:

```
npx skills add ../acme-skills --skill changelog-lint -g -a claude-code codex -y
```

With two target directories, the CLI uses symlink mode, and the result block reports `universal: Codex` and `symlinked: Claude Code`. The real copy is `~/.agents/skills/changelog-lint`, where Codex reads it, and `~/.claude/skills/changelog-lint` is a link to it. The install is recorded in the global lock file, not the project's.

Without `-a`, project-only agents report a per-agent failure (Universal and agent-specific agents).

Add an agent to an installed skill

To give an installed skill to another agent, run `add` again and list every agent that should have it, including those that already do:

```
npx skills add ../acme-skills --skill release-notes -a claude-code codex -y
```

The installation summary's `overwrites: Claude Code` line names targets whose files the CLI replaces. With two target directories, the CLI writes the real copy to `.agents/skills`, where Codex reads, and replaces Claude Code's copy with a link to it. Running with only `-a codex` would target one directory, `.agents/skills`, so the CLI would copy the files there and leave Claude Code's separate copy in place. The two copies would then drift apart.

Check for and apply updates

`npx skills update` reinstalls skills whose source has changed. `-p` limits it to the project, and `-y` suppresses prompts. Run `npx skills update -p -y`. Because `release-notes` came from a local path, which `update` never checks, the command reports `No project skills to update.` Reinstall `release-notes` from the Git repository, and run `update` again:

```
npx skills add file:///tmp/demo/acme-skills.git --skill release-notes -a claude-code
codex -y
npx skills update -p -y
```

The update now ends with `✓ All project skills are up to date`.

Publish a change from a working clone, then update:

```
cd /tmp/demo
git clone -q acme-skills.git acme-work
echo "4. Link each entry to its pull request." >> acme-work/skills/release-
notes/SKILL.md
git -C acme-work commit -q -am "Link entries to pull requests"
git -C acme-work push -q
cd project
npx skills update -p -y
tail -n 1 .claude/skills/release-notes/SKILL.md
```

```
Checking for skill updates…
Updating for: Universal, Claude Code
Checking 1 skill(s)…
Updating release-notes…
  ✓ Updated release-notes
✓ Updated 1 skill(s)
4. Link each entry to its pull request.
```

The update reinstalls for the agents detected now, not those you named with `-a` (How updates are detected and applied).

> **IMPORTANT**
> In skills 1.7.4, `npx skills check` is an alias of `update`: it installs the changes it finds. Do not run `check` where you want only a report, such as a CI job that must not change files (`skills update` and `skills check`).

Update failures are listed in Errors when updating or restoring.

Remove skills

`npx skills remove` deletes skills from the project, or from the home directory with `-g`. Pass `-a` to remove a skill from one agent:

```
npx skills remove release-notes -a claude-code -y
```

Then run `npx skills list`:

```
Project Skills
release-notes ./.agents/skills/release-notes
  Agents: Codex  Source: file:///tmp/demo/acme-skills.git
```

The CLI deleted Claude Code's link but kept the canonical copy and its lock entry, because Codex, a detected agent, still reads the copy.

> **WARNING**
> Removing a skill from one agent also deletes the canonical copy in `.agents/skills` and the skill's lock entry unless another agent detected on this machine still has the skill. Links from agents the CLI does not detect are left pointing at nothing, and those agents lose the skill. Run `npx skills list` after removing from one agent, and rerun `add` for the agents that still need the skill (`skills remove`).

A universal agent reads the canonical copy directly, so `-a` cannot remove a skill from one universal agent while other agents still use it: with Claude Code also installed, `remove release-notes -a codex -y` reports success but leaves the copy and its lock entry in place. To keep a skill for Claude Code only, remove it from every agent, then reinstall with `-a claude-code`, which copies it into `.claude/skills`:

```
npx skills remove release-notes -y
npx skills add file:///tmp/demo/acme-skills.git --skill release-notes -a claude-code
```

```
-y
```

Without `-a`, as in the first command, `remove` deletes the skill from every agent directory, along with the canonical copy and the lock entry. `--all` removes every skill in the chosen scope and implies `-y`, as in `npx skills remove --all -g`.

Removing a name that matches no installed skill prints `No matching skills found for: <name>` and exits 0, so a script cannot detect a mistyped name from the exit status. For other surprises, see Unexpected placement and removal results.

Try a skill without installing it

`npx skills use` prints a prompt that tells an agent to follow one skill, without installing it:

```
npx skills use ../acme-skills --skill release-notes
```

The prompt is the only output on stdout and contains the whole `SKILL.md`. For a skill with supporting files, the prompt names a new directory under the system temporary directory that holds those files. The CLI leaves that directory in place after it exits, so delete it when you are done. `--agent` starts an agent with the prompt instead; in 1.7.4 it accepts only `claude-code`, `codex`, and `sarvam-code`.

CHAPTER 5

Sharing Skills Through a Project

A team shares a project's skills by committing its lock file, and optionally the installed files, to the repository. The restore and sync commands are experimental in skills 1.7.4 and can change in later releases; see `skills experimental_install` and `skills experimental_sync`.

Before you begin

- Read Lock files in Chapter 3.
- Build the sandbox in The sample setup, then run `mkdir -p ~/.claude` so that the CLI detects Claude Code.

In this chapter you will

- choose which installed files to commit;
- restore a project's skills in a fresh checkout;
- sync skills from npm dependencies, and declare skills in `package.json`.

Decide what to commit

Commit `skills-lock.json`. It is the only record of where each project skill came from.

Choose how teammates get the skill files:

Choice	A fresh clone contains	Consequence
Commit `.agents/skills` and agent link directories such as `.claude/skills`	The files you reviewed	The agent links resolve because their target is committed too.

Ignore both	Only the lock file	Each teammate restores, then adds agent links.

Restoring does not reproduce the files you reviewed. The lock file records a content hash but no commit, so a restore installs whatever each Git source's `ref`, or its default branch, holds at that moment. Commit the files, or pin a ref, when the team must run exactly what it reviewed.

> **WARNING**
> Do not commit the links that sync creates. They point into `node_modules`, so they dangle in a fresh clone until someone runs `npm install`. List them in `.gitignore`, and let sync recreate them.

Restore skills from the lock file

The first command installs both sample skills and records them in the lock file. The others delete the files and restore them:

```
npx skills add ../acme-skills --skill '*' -a claude-code codex -y
rm -rf .agents .claude
npx skills experimental_install
```

```
● Restoring 2 skills from skills-lock.json into .agents/skills/
◇ Found 2 skills
◆ Restored changelog-lint, release-notes
```

The command installs lock entries into `.agents/skills`, which universal agents read. It creates no agent-specific links, so `.claude/skills` is still missing; rerun the `add` command to create it.

Entries with source type `node_modules` go to sync, which links them from the installed packages, so run `npm install` first.

> **WARNING**
> If a synced package is missing from `node_modules`, sync deletes its skill's lock entry and still exits 0. Recover with `git checkout skills-lock.json`, run `npm install`, and restore again.

When one source fails, the command prints `Failed to install from <source>: …`, restores the remaining sources, and exits with status 1.

> **IMPORTANT**
> In skills 1.7.4, `npx skills install` is an alias of `add` and does not restore. Without a source, it fails with `Missing required argument: source` and exits 1.

Sync skills shipped in npm packages

An npm package can ship skills for the projects that depend on it. Sync scans each package listed in the project's `dependencies`, `devDependencies`, `optionalDependencies`, or `peerDependencies`. A package with a `SKILL.md` at its root is one skill. Otherwise sync reads every `skills/<name>/SKILL.md` and `dist/skills/<name>/SKILL.md` in it.

Give `acme-app` a dependency. Because `acme-deploy` is fictional, these commands stand in for `npm install`:

```
cat > package.json <<'EOF'
{
  "name": "acme-app",
  "private": true,
  "devDependencies": { "acme-deploy": "1.2.0" }
}
EOF
mkdir -p node_modules/acme-deploy/skills/deploy-preview
echo '{ "name": "acme-deploy", "version": "1.2.0" }' > node_modules/acme-
deploy/package.json
cat > node_modules/acme-deploy/skills/deploy-preview/SKILL.md <<'EOF'
---
name: deploy-preview
description: Deploy a preview build with acme-deploy.
---

Run `acme-deploy preview` and report the preview URL.
EOF
```

Preview the result with `npx skills experimental_sync --dry-run -y`, which changes nothing. Then sync and inspect the links:

```
npx skills experimental_sync -y
readlink .agents/skills/deploy-preview .claude/skills/deploy-preview
```

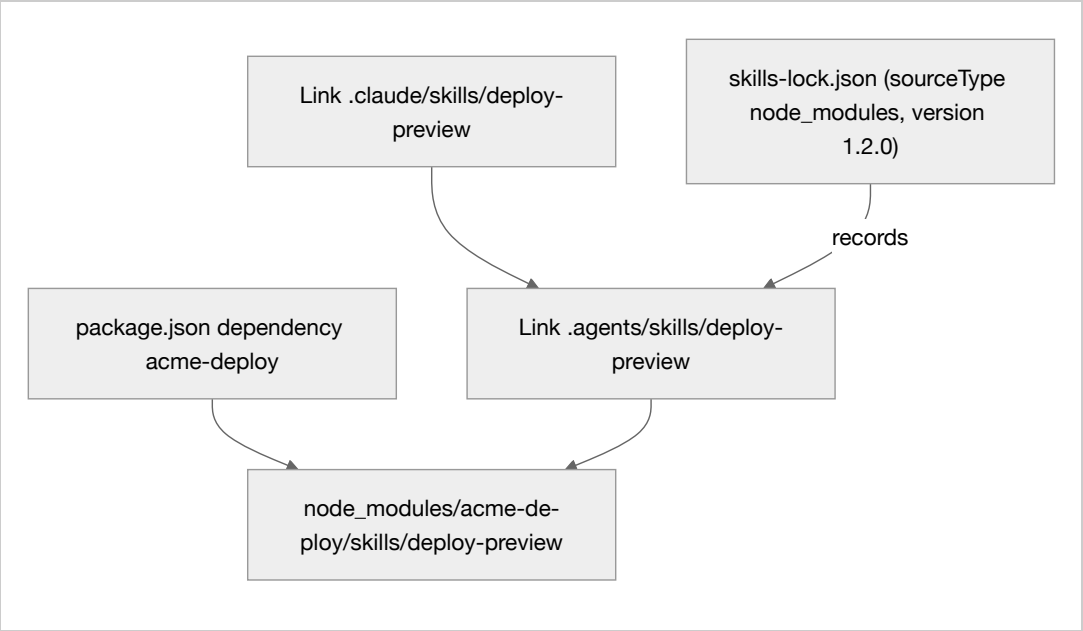
```
../../node_modules/acme-deploy/skills/deploy-preview
```

```
../../.agents/skills/deploy-preview
```

The lock entry records the package version:

```
"deploy-preview": {
  "source": "acme-deploy",
  "sourceType": "node_modules",
  "skillPath": "skills/deploy-preview/SKILL.md",
  "version": "1.2.0",
  "computedHash": "d499257bf750a7d5974b8256a115d507fa3824c1b653fb6b289ed97780c58172"
}
```

Figure 5-1: Synced skills are links into node_modules, so they change when the package changes.



Agents see a package upgrade as soon as npm writes the new files. Rerun sync after an upgrade to record the new version and to pick up skills the package added or dropped. If you pass `--copy`, sync copies the files instead of linking them, and agents see an upgrade only after the next sync.

Sync chooses agents as `add` does, except that with `-y` and no agent detected it installs for universal agents only. Inside a detected agent, it targets that agent and the universal agents.

Each run removes the skills that sync installed and that nothing provides anymore, with their links and lock entries. `--no-cleanup` keeps them, and their links dangle once the package is gone.

Declare skills in package.json

The `skills` array in `package.json` names skills to install without shipping their files. Sync reads it from the project and from each direct dependency. The skills-npm specification defines its grammar, and skills 1.7.4 accepts two kinds of entries:

- `npm:<package>` installs the skills an installed package ships, even when the package is not a direct dependency.
- A Git source is a string, or an object with `source`, an optional `skills` list of names to keep, and an optional `ref`. A local path or plain URL is rejected, and in the project's own field it stops sync with exit status 1.

This field installs `release-notes` from the sample repository at tag `v1.0.0`:

```
"skills": [
  {
    "source": "file:///tmp/demo/acme-skills.git",
    "skills": ["release-notes"],
    "ref": "v1.0.0"
  }
]
```

To try it, add the field to `package.json` and run:

1. `git -C /tmp/demo/acme-skills.git tag v1.0.0`, which tags the bare repository that the field names.
2. `npx skills remove release-notes -y`, because sync never replaces a skill that `add` installed.
3. `npx skills experimental_sync -y`, which clones the source at the tag and copies the skill into `.agents/skills`.

Sync fetches a Git entry only once. To pick up changes, run `npx skills update`. Update refreshes the files but records the skill as an `add` install, so sync no longer removes the skill when you delete it from the field.

A dependency can declare the field too, which makes the package a skill pack.

To narrow a sync, pass patterns to `--include` and `--exclude`. A pattern `pkg` matches every skill that package ships or declares, `pkg#skill` matches one of them, and `.` matches the Git entries in the project's own field. Both parts accept globs, and an exclusion wins. `skills experimental_sync` lists the other options.

CHAPTER 6

Authoring and Publishing Skills

This chapter is for people who write skills for others to install. It builds the `acme-skills` source that earlier chapters install from, tests it, and publishes it from a Git repository, a website, or an npm package. Read only the publishing section for your channel.

Before you begin

- Read The SKILL.md contract and How the CLI discovers skills in Chapter 2.
- Build the sandbox in The sample setup, leaving out `acme-skills`, which this chapter creates in `/tmp/demo`.

In this chapter you will

- create, fill in, and test a skill;
- publish it from Git, a website, or npm;
- hide a skill that is not ready.

Create a skill

Create the `skills` container directory, then run `skills init` with the skill's name:

```
$ mkdir -p acme-skills/skills && cd acme-skills/skills
$ npx skills init release-notes
Initialized skill: release-notes

Created:
  release-notes/SKILL.md
…
```

The template sets `name: release-notes`, a placeholder description, and placeholder headings and steps.

Run without a name, `init` writes `SKILL.md` into the current directory and takes the name from that directory, so `mkdir changelog-lint && cd changelog-lint && npx skills init` creates a skill named `changelog-lint`. If the target `SKILL.md` already exists, `init` prints `Skill already exists at release-notes/SKILL.md`, leaves the file unchanged, and exits with status 0.

Write the description and instructions

Replace the template with the real description and instructions, and add `template.md`, the file the instructions refer to. The sample setup shows both finished files.

Keep `name` identical to the directory name, and use only lowercase letters, digits, and single hyphens. The CLI names the installed directory after the sanitized `name`, not after the source folder (Skill names and install directories). When the two agree, the folder a consumer sees in your repository is the one that appears in their project.

Write the `description` to say what the skill does and when to use it. The CLI shows it beside the name in `--list` output and in the selection prompt, and the installed `SKILL.md` carries it to the agent.

Refer to supporting files by paths relative to the skill directory, as the instructions do with `template.md`. The CLI installs the whole directory, so the path resolves the same way after installation.

Test a skill before publishing

From `/tmp/demo`, run `npx skills add ./acme-skills --list` to see the source as a consumer would. It prints each skill's name and description. A skill missing from the list was skipped, and the warning above the list gives the reason (The SKILL.md contract). Fix every warning before you publish, because consumers never see a skipped skill.

Next, from `project/`, run `npx skills use ../acme-skills --skill release-notes` to print the exact prompt an agent would receive (Try a skill without installing it). Then install the skill into the scratch project with `npx skills add ../acme-skills --skill release-notes -a claude-code -y`, and confirm that `.claude/skills/release-notes` holds both `SKILL.md` and `template.md`.

Lay out a repository of skills

Put each skill in its own directory under `skills/`, as `acme-skills` does with `skills/release-notes/SKILL.md`. You can group skills in category folders, such as `skills/release/release-notes/`, within the depth that discovery walks. Do not put a `SKILL.md` at the root of a repository that holds several skills. A root skill hides every other skill unless consumers pass `--full-depth`. To present skills in named groups, declare them in a plugin manifest. How the CLI discovers skills gives the rules for all three.

Publish from a Git repository

Commit the skills, tag the release, and push both:

```
git tag v1.0.0
git push origin main v1.0.0
```

Consumers install from `acme/acme-skills`, select one skill with `acme/acme-skills@release-notes` or a tree URL such as `https://github.com/acme/acme-skills/tree/v1.0.0/skills/release-notes`, or pin the release with `acme/acme-skills#v1.0.0` (Source formats).

Tag every release so that consumers can pin one (Pinning a ref and naming a skill). Consumers pin a tag to keep the content they reviewed. `update` rechecks the recorded ref, so moving the tag makes their next update install content they never reviewed. Publish a new tag instead.

Consumers clone a private repository with the Git credentials, GitHub CLI login, or SSH keys they already have (Network access, authentication, and telemetry).

Publish from a website

A website can publish a catalog through a well-known index. Serve `index.json` at `/.well-known/agent-skills/index.json`; the CLI also reads the older `/.well-known/skills/index.json`. The Agent Skills site defines the schema. How the CLI validates an index depends on its `$schema` field:

- With the v0.2.0 schema URL, each entry names one artifact by `type`, `url`, and SHA-256 `digest`. The CLI drops, without a warning, any entry with an invalid name, description, `type`, `url`, or digest, and any skill whose downloaded artifact does not match its digest.
- Without `$schema`, the index uses the older format that lists each skill's files, and one invalid entry makes the CLI reject the whole index.
- With any other `$schema`, the CLI ignores the index.

A valid name has 1 to 64 lowercase letters, digits, and hyphens, with no leading, trailing, or doubled hyphen. Run `npx skills add https://acme.dev --list` after each deploy and confirm that every skill appears.

A consumer who adds a URL with a path, such as `https://acme.dev/team/docs`, gets only an index under that path. When none exists there but the host has a root index, the command fails with `No skills found for the scoped path '/team/docs'` instead of installing the host's whole catalog.

A website can also offer one file for direct download: a `SKILL.md`, a ZIP archive, or a tar archive, gzipped or not. If your host serves a root well-known index, a direct-download URL under a path on that host fails with the scoped-path error. Either list the skill in the index, or serve direct downloads from a host that has no index.

Keep direct downloads within these limits, or tell consumers which variable to set to a larger byte or file count:

Limit	Default	Variable
Download size	10 MiB	`SKILLS_DOWNLOAD_MAX_BYTES`
Extracted size	25 MiB	`SKILLS_EXTRACT_MAX_BYTES`
Files in an archive	1,000	`SKILLS_EXTRACT_MAX_FILES`

Ship skills in an npm package

Put the skills in `skills/<name>/` or `dist/skills/<name>/` in the package, one level deep. If the package has a `files` list, include that directory:

```
{
  "name": "acme-skills",
  "version": "1.0.0",
  "files": ["skills"]
}
```

Run `npm pack --dry-run` and confirm that each `SKILL.md` and supporting file appears in the list. Consumers install the package and run sync, as Sync skills shipped in npm packages describes.

Hide work-in-progress skills

To publish a skill without offering it to everyone, add this to its frontmatter:

```
metadata:
  internal: true
```

Internal skills appear only when named (Selecting skills to install). Anyone who can read the source can still read and install it.

CHAPTER 7

Command Reference

Look up a command here; the task chapters show each one in use.

Conventions for every command

Arguments and flags

- `-a`, `-s`, and `--subagent` accept several values. Each consumes the following words until the next word that starts with `-`, so a source written after one becomes a value: `npx skills add -a claude-code ../acme-skills -y` fails with "Missing required argument: source". Put the source first.
- `'*'` as the value of `-a` or `-s` means every agent or every skill. Quote it so that the shell does not expand it.
- Most commands ignore flags they do not recognize, without a warning. `use` rejects them, and `find` treats them as query words.
- `--help` or `-h` after any command prints help and exits 0 without running the command.
- `npx skills --version` prints the version.

Non-interactive mode and agent detection

The CLI prompts only when standard input is a terminal (TTY). Without one, a prompt is cancelled: `add` exits 1 with "Interactive prompt required but stdin is not a TTY", `remove` exits 0 without changing anything, and `experimental_sync` exits 0 after its cleanup has already run (`skills experimental_sync`). Pass `-y` (`--yes`) to skip prompts and use the defaults.

The CLI detects that a coding agent is running it from environment variables that agents set, such as `CLAUDECODE`, `CODEX_THREAD_ID`, and `AI_AGENT`. Inside a detected agent, `add`, `remove`, and `experimental_sync` behave as if you passed `-y`, and `add` and `experimental_sync` target that agent plus the universal agents unless `-a` names agents. Pass `-a` in scripts so that the targets do not depend on the environment that runs them; How the CLI chooses target agents gives the full order.

Output streams and exit statuses

Human-readable output and most errors go to standard output. `add --json` reserves standard output for exactly one JSON array, even on failure, and sends everything else to standard error.

Commands exit 1 for:

- a missing or unusable source: no argument, a local path that does not exist, a failed clone, or no valid skills;
- `add --skill` names that match nothing, unknown agent names, and invalid option values;
- failed updates and restores, and `add --json` entries that are `failed` or `skipped`;
- unknown commands.

These cases exit 0 although work was not done:

- `add` without `--json` when installation fails for some or all target agents;
- `remove` and `update` with names that match no installed skill;
- `find`, even when the search or an option fails.

Network access, authentication, and telemetry

A Git source is cloned with `git clone` and your existing Git configuration, so credential helpers and SSH keys apply. If cloning a GitHub HTTPS URL fails authentication, the CLI retries with `gh repo clone`, and then with the SSH URL in batch mode, which fails rather than prompting for a passphrase.

GitHub API requests go out anonymously first. After a rate limit or an authorization failure, the CLI retries with one token, `GITHUB_TOKEN` or, if that is unset, `GH_TOKEN`, and then with `gh api`. The CLI does not read the token that the GitHub CLI stores; only `gh` itself uses it.

The CLI sends usage events to skills.sh for installs, removals, updates, searches, and syncs. For a GitHub source, `add` names the repository and skills in its install event, and in its request to the skills.sh audit service, only after GitHub confirms that the repository is public. Two kinds of event include source and skill names without that check: install events for website sources and for Git paths deeper than `owner/repo`, and remove events for every recorded source. Set `DISABLE_TELEMETRY=1` or `DO_NOT_TRACK=1` to stop both telemetry and audit requests. src/telemetry.ts defines each event.

`skills add`

Installs skills from a source for the chosen agents and records them in a lock file. Aliases: `a`, `install`, `i`.

```
npx skills add <source> [options]
```

Options

Flag	Value	Default	Meaning
`-g, --global`		project	Install under the home directory.
`-a, --agent`	agents, or `'*'`	detected	Target agents.
`-s, --skill`	names, or `'*'`	prompt	Skills to install. With `-y`, all.
`-l, --list`		off	List the source's skills without installing.
`-y, --yes`		off	Skip prompts.
`--all`		off	Same as `-s '*' -a '*' -y`.
`--copy`		off	Use copy mode.
`--full-depth`		off	Search below a root `SKILL.md`.
`--json`		off	Print results as JSON. Requires `-y` or `--all`; not with `--list` or website sources.
`--metadata`	JSON	none	Attach the JSON to the install telemetry event.
`--subagent`	names	root	Install into these Eve subagents (`root` for the root agent).

Each `--json` entry has `name` and `status`: `installed`, `skipped` (a `--skill` name matched nothing, with a `reason`), or `failed` (some target agent failed, with an `error`). A failure before skill selection, such as a missing path, gives one entry with only `status: "failed"` and `error`, and no `name`. Installed entries add `source`, `ref`, `hash` (the lock's `computedHash`), `path`, `scope`, `agents`, `mode`, and `security` (audit results).

The exit status follows Output streams and exit statuses.

Behavior

- Discovers skills as How the CLI discovers skills describes, writes them to the canonical directory and agent directories, and records them in the lock file for the scope.
- Rerunning an install replaces the files and the lock entry. With the same arguments, including `-a`, it reaches the same result, but it discards local edits to the installed files.

Install a skill into a project shows output; Errors when adding skills explains failures.

`skills use`

Prints a prompt that hands one skill to an agent, without installing it.

```
npx skills use <source>[@<skill>] [options]
```

Options

Flag	Value	Default	Meaning
`-s`, `--skill`	one name	none	The skill to use.
`-a`, `--agent`	one agent	none	Start `claude-code`, `codex`, or `sarvam-code` with the prompt.
`--full-depth`		off	Search below a root `SKILL.md`.

Behavior

- Requires exactly one skill; a source with several fails and lists them.
- Copies the skill to a temporary directory that persists after exit, and prints the `SKILL.md` text on standard output, with that directory's path when the skill has supporting files.
- Writes nothing to the project or home directory. Exits 1 on error; with `-a`, exits with the agent's status.

Try a skill without installing it shows the prompt.

`skills list`

Lists installed skills with their agents and sources. Alias: `ls`.

```
npx skills list [options]
```

Options

Flag	Value	Default	Meaning
`-g, --global`		project	List global skills.
`-a, --agent`	agents	all	Check only these detected agents when filling each skill's `Agents:` line. Skills they lack still appear, as `not linked`.
`--json`		off	Print a JSON array.

`-a` narrows the JSON `agents` array the same way. JSON entries have `name`, `path`, `scope`, `agents`, and the lock's `source`, `sourceUrl`, and `sourceType` fields. The command changes nothing and exits 1 only for an unknown agent. List installed skills shows both formats.

`skills remove`

Deletes installed skills. Aliases: `rm`, `r`.

```
npx skills remove [<name>...] [options]
```

Options

Flag	Value	Default	Meaning
`-g, --global`		project	Remove global skills.
`-a, --agent`	agents	every agent	Remove only these agents' entries. `'*'` is rejected.
`-s, --skill`	names, or `'*'`	prompt	Same as positional names.
`-y, --yes`		off	Skip confirmation.
`--all`		off	Remove every skill; implies `-y`. Exits 1 if names are also given.

Behavior

- A name matches an installed skill, or else exactly matches a lock entry's source (absolute for local paths) and selects all of that source's skills.

- Without `-a`, deletes the skill from every agent directory, the canonical directory, and the lock file.
- With `-a`, keeps the canonical copy and the lock entry only while an unnamed agent that the CLI detects on this machine still has the skill. Otherwise it deletes both, leaving other agents' links dangling; rerun `add` to repair them.
- Exits 1 for an unknown agent or `--all` with names.

Remove skills shows each form.

`skills update` and `skills check`

Reinstalls tracked skills whose source changed. Aliases: `upgrade`, `check`.

```
npx skills update [<name>...] [options]
```

> **IMPORTANT**
> **Current state (skills 1.7.4):** `check` runs `update`, so it installs changes instead of only reporting them.

Options

Flag	Value	Default	Meaning
`-g, --global`		resolved	Update global skills.
`-p, --project`		resolved	Update project skills.
`-y, --yes`		off	Resolve the scope without a prompt.

Behavior

The command chooses the scope from the flags, the names, and whether it can prompt:

Flags	Names given	–y or no TTY	Scope
`-g`	either	either	global
`-p`	either	either	project
`-g` and `-p`	no	either	both
`-g` and `-p`	yes	either	global

neither	yes	either	both
neither	no	yes	project if the directory has `skills-lock.json` or a skill in `.agents/skills`, else global
neither	no	no	prompt

- Reinstalls changed skills as How updates are detected and applied describes.
- Skips local-path and npm-package entries.
- Exits 1 if any skill fails to update.

Check for and apply updates shows both outcomes.

`skills find`

Searches skills.sh for skills. Aliases: `search`, `f`, `s`.

```
npx skills find [<query>] [--owner <owner>]
```

`--owner` limits results to one GitHub owner's repositories. With a query, the command prints up to 20 results as `owner/repo@skill` lines that `add` accepts, with links to skills.sh. Without a query, it opens an interactive search and installs the chosen skill; without a TTY, or inside an agent, it prints usage instead. `SKILLS_API_URL` replaces the default search server, `https://skills.sh`. It exits 0 even when the search fails.

`skills init`

Creates `<name>/SKILL.md`, or `./SKILL.md` named after the current directory, from a template. It leaves an existing file untouched and exits 0. Create a skill shows the template.

```
npx skills init [<name>]
```

`skills experimental_install`

Restores the skills in `skills-lock.json`. This command is experimental and can change in later releases.

```
npx skills experimental_install
```

It installs each entry into `.agents/skills` only, creating no agent-specific links, and syncs npm-package entries from `node_modules`. A failing source does not stop the others but sets exit status 1. Restore skills from the lock file shows a restore.

`skills experimental_sync`

Links skills shipped by npm dependencies into the project. This command is experimental and can change in later releases. Alias: `sync`.

```
npx skills experimental_sync [options]
```

Options

Flag	Value	Default	Meaning
`-a, --agent`	agents, or `'*'`	detected	Target agents.
`-y, --yes`		off	Skip prompts; with no agent detected, target universal agents.
`--copy`		off	Copy instead of linking into `node_modules`.
`--dry-run`		off	Report changes without making them.
`--no-cleanup`		off	Keep skills no dependency ships.
`--no-remote`		off	Skip Git sources in `package.json` `skills` fields.
`-r, --recursive`		off	Include workspace packages' dependencies.
`--include`	patterns	all	Sync only `pkg` or `pkg#skill` matches.
`--exclude`	patterns	none	Skip matching packages or skills.

The command works in project scope only and keeps skills installed with `add`. Its cleanup runs before the `Proceed with sync?` prompt, so a declined or cancelled run has still removed skills that no dependency provides, with their links and lock entries. Pass `--dry-run` or `--no-cleanup` to delete nothing. It exits 1 for an invalid `skills` field, an unknown agent, or a failed install. Sync skills shipped in npm packages shows a sync.

APPENDIX A

Files and Environment Variables

About Scopes, Agents, and Lock Files explains the model behind these files.

File locations

Paths without `~` are relative to the project directory, the current working directory.

Path	Scope	Written by	Purpose
`.agents/skills/<name>/`	Project	`add`, `update`, `experimental_install`, `experimental_sync`	Canonical copy, read by universal agents
Agent-specific directories, such as `.claude/skills/` (all agents)	Project or global	`add`, `update`, `experimental_sync`	Link to the canonical copy, or an independent copy
`skills-lock.json`	Project	`add`, `remove`, `update`, `experimental_install`, `experimental_sync`	Project lock file; commit it
`~/.agents/skills/<name>/`	Global	`add -g`, `update -g`	Global canonical copy
`~/.agents/.skill-lock.json`, or `$XDG_STATE_HOME/skills/.skill-lock.json` when that variable is set	Global	`add -g`, `remove -g`, `update -g`; interactive `add` and `experimental_sync` in a project (remembered agent choices and dismissed prompts)	Global lock file

Project lock file fields

`skills-lock.json` maps each skill name to an entry with these fields:

- `source`: what to pass to `add` to reinstall.
- `sourceUrl`: the clone URL of a `git` or `gitlab` source. `experimental_install` cannot restore a generic Git entry without it.
- `sourceType`: how the skill is fetched. `update` never updates `local` or `node_modules` entries in place.
- `ref`: the pinned branch or tag, which update and restore fetch again.
- `skillPath`: the skill's `SKILL.md` within the source. `update` cannot update a remote entry without it.
- `computedHash`: SHA-256 of the source skill folder's file paths and contents. `update` compares it with the source, so edits to installed files go undetected.
- `version`: the package version that shipped a `node_modules` skill.
- `via`: the package whose `package.json` `skills` field requested the skill (`.` is the project). Sync removes the entry when the request goes away, so edit `package.json`, not the lock.

`src/local-lock.ts` defines every field.

Global lock file

The global lock (its path is in the last row of the table above) has `version` 3. The CLI treats a lock with a lower version as empty and replaces it at the next write, including one from an interactive project-scope `add`. Skills it listed stay installed, but `update -g` ignores them until you reinstall them with `add -g`.

Apart from well-known entries, `update -g` checks an entry only when it has both `skillFolderHash` and `skillPath`. It lists other entries, including every local one, as "cannot be checked automatically", with an `add` command to rerun. `src/skill-lock.ts` defines the remaining fields.

Environment variables

Variable	Effect	Default
`DISABLE_TELEMETRY`	Any non-empty value stops telemetry and audit requests (details)	Unset
`DO_NOT_TRACK`	Same as `DISABLE_TELEMETRY`	Unset
`INSTALL_INTERNAL_SKILLS`	`1` or `true` shows internal skills	Unset
`GITHUB_TOKEN`	Token for GitHub API requests that fail anonymously	Unset

`GH_TOKEN`	Used when `GITHUB_TOKEN` is unset	Unset
`GH_HOST`	GitHub Enterprise host for shorthand sources, cloned as `git`	`github.com`
`XDG_STATE_HOME`	Moves the global lock	Unset
`XDG_CONFIG_HOME`	Base of some agents' global directories, such as Goose's	`~/.config`
`CLAUDE_CONFIG_DIR`, `CODEX_HOME`, and other agent homes	Moves an agent's home, used for detection and as the parent of an agent-specific global `skills/`	`~/.claude`, `~/.codex`, …
`SKILLS_DOWNLOAD_MAX_BYTES`	Largest direct download	10 MiB
`SKILLS_EXTRACT_MAX_BYTES`	Largest total extracted size	25 MiB
`SKILLS_EXTRACT_MAX_FILES`	Most files in an archive	1000
`SKILLS_CLONE_TIMEOUT_MS`	Git clone time limit	300000 (5 minutes)
`SKILLS_API_URL`	Search API base for `find`	`https://skills.sh`

Agent-detection variables are covered in Non-interactive mode and agent detection.

APPENDIX B

Errors and Recovery

This appendix indexes the messages and surprising results of skills 1.7.4. A source or selection error writes nothing, while a per-agent failure leaves the other agents' installs in place. `skills add` says when a rerun is safe, and Output streams and exit statuses lists exit statuses.

Errors when adding skills

Message or symptom	Cause	Recovery	See
`No valid skills found…`, often after `⚠ Skipped <path> — missing required frontmatter field(s): description`	No `SKILL.md` has string `name` and `description` fields.	Fix the files named, or point at the skills' directory.	Contract
`No matching skills found for: <name>`	No skill has that `name`. `add` exits 1, and `remove` exits 0.	Use a listed name (case-insensitive).	Selection
`Invalid agents: <names>`	An –a value is not an agent ID.	Use an ID from `Valid agents:`.	Targets
`Local path does not exist: <path>`	The path is wrong, or ends in `@skill`.	Fix the path, and name skills with `--skill`.	Naming
`Failed to clone repository`	Git failed; its error follows.	Fix the URL or credentials.	Authentication
`Interactive prompt required but stdin is not a TTY`	A prompt needed a terminal. Nothing was installed.	Pass –a and –y.	Prompts
`The --json flag requires --yes (or --all)…`	`--json` never prompts.	Add –y.	`skills add`
`<Agent> does not support global skill installation`	The agent is project-only. Other agents still install.	Install for it in project scope.	Agents

`Symlinks failed for: <agents>`	Linking failed, so those agents got copies.	On Windows, enable Developer Mode and rerun.	Modes

Errors when updating or restoring

Message or symptom	Cause	Recovery	See
`No project skills to update.`, or `N skill(s) cannot be checked automatically:` with `(Local path)`	`update` skips local-path and npm-synced skills.	Rerun the original `add` (shown after `To update:` for global skills), or `experimental_sync`.	Updates
`✗ Failed to update <skill>`	The hidden reinstall failed, as it does in 1.7.4 for subdirectory skills of a `file://` source not ending in `.git`.	Rerun `add` with the recorded source.	Updates
`Cannot restore <skill>: skills-lock.json is missing sourceUrl…`	An older generic Git or GitLab entry lacks its URL, so the skill is skipped.	Rerun `add` with the full URL.	Restore
`Missing required argument: source` from `install`	`install` aliases `add`.	Run `experimental_install`.	Restore

Unexpected placement and removal results

Message or symptom	Cause	Recovery	See
Every agent got the skill; `.claude/` and `agent/` appeared	–y without –a found no usable agent.	Remove it, and reinstall with –a.	Targets
`skipped: <agents> (project directory not found)`	The agent's directory, such as `.windsurf/`, is absent.	Create it or use –a, then rerun `add`.	Targets
The skill landed in a subdirectory	Project scope is the current directory.	Remove it, and rerun `add` from the project root.	Scopes
`(copied)` without `--copy`	All targets share one directory.	None needed.	Modes
An installed copy no longer matches its source	Edits to the source, or to one copy, do not spread.	Rerun `add`, which discards local edits.	`skills add`

Dangling links after `remove -a`	No other detected agent had the skill, so the canonical copy was deleted.	Rerun `add` for the agents that need it.

APPENDIX C

Glossary

agent: a coding tool, such as Claude Code or Codex, that reads installed skills. Compare *target agent*.

agent-specific agent: an agent with its own skills directory, such as `.claude/skills`, which receives a link or a copy. Compare *universal agent*.

canonical directory: `.agents/skills`, or `~/.agents/skills` globally, holding each skill's one real copy in symlink mode.

computed hash: the project lock's SHA-256 hash of a skill folder's contents. Compare *skill folder hash*.

copy mode: placement giving each target agent its own copy, chosen by `--copy` or when all targets share one directory. Compare *symlink mode*.

direct download: a source that is one `SKILL.md` or archive at a URL. The project lock omits it, so it cannot be updated or restored.

discovery: the search for skills inside a source. A `SKILL.md` at the search root ends it unless you pass `--full-depth`.

frontmatter: the YAML block that opens `SKILL.md`. A skill lacking string `name` and `description` fields is skipped with a warning.

global lock file: `~/.agents/.skill-lock.json` by default, recording `-g` installs. Compare *project lock file*.

global scope: installation under the home directory with `-g`. Compare *project scope*.

install directory: the directory a skill installs into, named after its sanitized frontmatter `name`. Same-named skills share one, so the later install overwrites the earlier. Compare *skill name*.

internal skill: a skill marked `metadata.internal: true`, hidden unless you name it.

packaged skill: a skill shipped in an npm package and linked from `node_modules` by sync, so it changes with the package.

plugin manifest: `.claude-plugin/marketplace.json` or `plugin.json`, whose declared skill directories discovery also searches.

project lock file: `skills-lock.json`, which records project installs for restores and updates. Commit it.

project scope: the default scope, rooted at the current directory, not the repository root.

ref: a Git branch or tag, given with `#ref` or a tree URL, that updates and restores fetch again.

skill: a directory containing `SKILL.md` and any supporting files, installed whole so relative references still resolve.

skill folder hash: the global lock's `skillFolderHash`, a GitHub tree SHA for GitHub sources. `update -g` checks only entries that have one. Compare *computed hash*.

skill name: the frontmatter `name`, which `--skill` matches. Compare *install directory*.

skill pack: a package whose `package.json` `skills` field declares skills for sync. Compare *packaged skill*.

SKILL.md: the file that makes a directory a skill: frontmatter, then instructions.

source: the `add` or `use` argument naming where skills live: a local path, repository, or URL.

source type: the kind of source recorded in `sourceType`, such as `local` or `github`. It controls fetching and whether `update` checks it.

supporting files: files beside `SKILL.md`, referenced by relative path, that install with the skill.

symlink mode: the default placement: one real copy in the canonical directory, linked from each agent-specific directory, so an edit reaches every linked agent.

sync: `experimental_sync`, which installs skills that npm dependencies ship or declare and removes stale ones.

target agent: an agent receiving a skill in one install. Without `-a`, detection chooses, so pass `-a` in scripts.

universal agent: an agent that reads `.agents/skills` directly, so it needs no link.

well-known index: an `index.json` under `/.well-known/agent-skills/` listing a website's skills, which the CLI tries before downloading a URL directly.